COURSEWORK ASSIGNMENT SUBMISSION

Name: Daksh Srivastava **Roll No:** 150096725087
Subject: Software Engineering & Project Management **Day:** Day 30 — Course Consolidation, Lessons Learned & Viva Preparation

---

Section 12: Final Coursework Reflection & Viva Preparation

Coursework consolidation, retrospective lessons learned, and technical viva defense preparation for the College Event Management System (CEMS).

1. Retrospective Lessons Learned (Engineering Reflection)

- **Lesson 1 (Requirements Rigor):** Vague requirements cause compounding architectural rework. In Week 1, defining registration capacity as "fast and scalable" without exact concurrency metrics delayed database locking implementation. Concrete Improvement: Mandate testable metrics and acceptance criteria for all user stories before writing code.
- **Lesson 2 (Concurrency Testing Early):** Functional unit tests with a single user thread completely masked a critical race condition where two simultaneous requests claimed the final remaining event seat. Concrete Improvement: Incorporate automated multi-threaded stress tests into the continuous integration pipeline from the first sprint.
- **Lesson 3 (Traceability Prevents Scope Creep):** Directly linking customer change requests to Git branches and Jira issues prevented uncontrolled feature creep and allowed precise estimation of delivery impact. Concrete Improvement: Enforce formal Change Control Board approvals for all mid-sprint scope modifications.

2. Comprehensive Five-Question Technical Viva Practice Sheet

Viva Question	Technical Response & Concrete Project Justification
Q1: Why did you choose a Relational Database (PostgreSQL) instead of a NoSQL document database for CEMS?	CEMS requires strict seat capacity enforcement where seat counter decrements must be atomic and serializable. PostgreSQL provides native ACID transaction guarantees and row-level locking (`SELECT ... FOR UPDATE`), preventing race conditions when multiple users attempt to register for the final remaining seat. NoSQL document stores require complex multi-document transaction handling to guarantee this level of consistency.
Q2: What is the technical difference between Verification and Validation, and how did you implement both?	Verification asks "Are we building the product right?" (checking conformance to design specifications via unit tests, static code analysis, and design reviews). Validation asks "Are we building the right product?" (checking if software fulfills customer operational needs via user acceptance testing, pilot festival trials, and feedback reviews). Both were tracked using our Requirements Traceability Matrix.

Q3: Explain how you calculated the Critical Path in your project schedule and what total float signifies.	The Critical Path was determined using forward and backward passes across our activity network (A to G). The critical path A-C-E-F-G totaled 14 days, representing the longest sequence of dependent activities. Total float ($LS - ES$) represents the duration an activity can slip without delaying project completion. Critical path activities have zero float.
Q4: What is Brooks's Law, and how did you prevent it from disrupting your sprint deadlines?	Brooks's Law states that "adding manpower to a late software project makes it later" due to training drag and quadratic growth in communication channels ($n(n-1)/2$). When our backend was delayed by 2 days, instead of adding new developers, we fast-tracked non-dependent testing activities and reduced scope on non-essential Sprint 1 backlog items (deferring advanced analytics).
Q5: How does Behaviour-Driven Development (BDD) bridge the gap between business stakeholders and QA engineers?	BDD utilizes human-readable Gherkin syntax (`Given-When-Then`) to formulate observable acceptance criteria. Business users understand the operational rules, developers implement exact domain boundaries, and QA testers execute automated cucumber/manual test cases directly mapped to the scenarios, eliminating ambiguity.

3. Final Handover Note for Future Maintenance Engineers

System Architecture Overview: CEMS Release 1.0 operates as containerized Docker microservices orchestrated via Docker Compose. The React SPA frontend is served via Nginx on port 80/443; the Node.js API runs on port 5000; PostgreSQL 16 runs on port 5432; Redis message queue runs on port 6379.

Deployment & Build Commands:

1. Initialize environment: `cp .env.example .env && docker compose up -d`
2. Run automated database migrations: `npm run migrate:latest`
3. Execute regression test suite: `npm run test:coverage` (assert >= 85% coverage)

Key Operational Cautions: Ensure Redis background workers are monitored; never run raw SQL updates on `registrations` table without updating `current_count` on `events` table to preserve relational invariant.